

Relationship Between Pointers and Arrays

- Arrays and pointers closely related
 - Array name like constant pointer
 - Pointers can do array subscripting operations
- Accessing array elements with pointers
 - Element **b[n]** can be accessed by ***(bPtr + n)**
 - Called pointer/offset notation
 - Addresses
 - **&b[3]** same as **bPtr + 3**
 - Array name can be treated as pointer
 - **b[3]** same as ***(b + 3)**
 - Pointers can be subscripted (pointer/subscript notation)
 - **bPtr[3]** same as **b[3]**

```

1 // Fig. 5.20: fig05_20.cpp
2 // Using subscripting and pointer notations with arrays.
3
4 #include <iostream>
5
6 using std::cout;
7 using std::endl;
8
9 int main()
10 {
11     int b[] = { 10, 20, 30, 40 };
12     int *bPtr = b;    // set bPtr to point to array b
13
14     // output array b using array subscript notation
15     cout << "Array b printed with:\n"
16         << "Array subscript notation\n";
17
18     for ( int i = 0; i < 4; i++ )
19         cout << "b[" << i << "] = " << b[ i ] << '\n';
20
21     // output array b using the array name and
22     // pointer/offset notation
23     cout << "\nPointer/offset notation where "
24         << "the pointer is the array name\n";
25

```

Using array subscript notation.

```

26  for ( int offset1 = 0; offset1 < 4; offset1++ )
27      cout << "*(b + " << offset1 << ") = "
28          << *( b + offset1 ) << '\n';
29
30  // output array b using bPtr and array subscript notation
31  cout << "\nPointer subscript notation\n";
32
33  for ( int j = 0; j < 4; j++ )
34      cout << "bPtr[" << j << "] = " << bPtr[ j ] << '\n';
35
36  cout << "\nPointer/offset notation\n";
37
38  // output array b using bPtr and pointer/offset notation
39  for ( int offset2 = 0; offset2 < 4; offset2++ )
40      cout << "*(bPtr + " << offset2 << ") = "
41          << *( bPtr + offset2 ) << '\n';
42
43  return 0; // indicates successful termination
44
45 } // end main

```

Using array name and
pointer/offset notation.

Using pointer subscript
notation.

Using **bPtr** and
pointer/offset notation.

Array b printed with:

Array subscript notation

b[0] = 10

b[1] = 20

b[2] = 30

b[3] = 40

Pointer/offset notation where the pointer is the array name

*(b + 0) = 10

*(b + 1) = 20

*(b + 2) = 30

*(b + 3) = 40

Pointer subscript notation

bPtr[0] = 10

bPtr[1] = 20

bPtr[2] = 30

bPtr[3] = 40

Pointer/offset notation

*(bPtr + 0) = 10

*(bPtr + 1) = 20

*(bPtr + 2) = 30

*(bPtr + 3) = 40

Pointers and Arrays

```
int * ptr; /* pointer to int (or int array) */  
  
char *chptr; /*pointer to char(or char array)*/  
  
int table[3]= {5, 6, 7}; /* array */  
  
ptr = table; //&table[0]//*assign array address  
to pointer*/  
  
cout << *ptr; /*prints 5*/  
cout << *(ptr+1); /* prints 6 */
```

Multi Dimensional Arrays (Matrices)

- An array can have two or more dimensions. This permits them to model multidimensional objects, such as graph paper or the computer display screen.
- They are often used to represent **tables of values** consisting of information arranged in **rows** and **columns**.

Pointers to 2D Arrays

$b[5] \Rightarrow *(b+5)$

`int a[5][5];`

$a[2][2] \Rightarrow *(*a+2)+2)$

```
const int ROWS = 3;
const int COLS = 3;
int main ()
{
    int arr[ROWS][COLS] = {1,2,3} , {2,4,6} ,
                           {3,6,9}};

    for (int i = 0; i < ROWS; i++)
    {
        for(int j = 0; j < COLS; j++)
            cout << *((arr+i)+j) << " ";
        cout << endl;
    }
    return 0;
}
```


Equivalent Expressions

`int a[n][m];`

- `a[n][m]`
- `*(* (a+n)+m)`
- `*(a[n]+m)`
- `(* (a+n))[m]`

Static & Automatic Local Arrays

□ Recall static storage

- If **static**, local variables save values between function calls
- Visible only in function body
- Can declare local arrays to be static
 - Initialized to zero

```
static int array[3];
```

□ If not static

- Created (and destroyed) in every function call

```
1 // Fig. 4.13: fig04_13.cpp
2 // Static arrays are initialized to zero.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 void staticArrayInit( void );      // function prototype
9 void automaticArrayInit( void );   // function prototype
10
11 int main()
12 {
13     cout << "First call to each function:\n";
14     staticArrayInit();
15     automaticArrayInit();
16
17     cout << "\n\nSecond call to each function:\n";
18     staticArrayInit();
19     automaticArrayInit();
20     cout << endl;
21
22     return 0; // indicates successful termination
23
24 } // end main
25
```

```
26 // function to demonstrate a static
27 void staticArrayInit( void )
28 {
29     // initializes elements to 0 first time function is called
30     static int array1[ 3 ];
31
32     cout << "\nValues on entering staticArrayInit:\n";
33
34     // output contents of array1
35     for ( int i = 0; i < 3; i++ )
36         cout << "array1[" << i << "] = " << array1[ i ] << " ";
37
38     cout << "\nValues on exiting staticArrayInit:\n";
39
40     // modify and output contents of array1
41     for ( int j = 0; j < 3; j++ )
42         cout << "array1[" << j << "] = "
43             << ( array1[ j ] += 5 ) << " ";
44
45 } // end function staticArrayInit
46
```

Static array, initialized to zero on first function call.

Array data is changed; the modified values stay.

```

47 // function to demonstrate an automatic local array
48 void automaticArrayInit( void )
49 {
50     // initializes elements each time function is called
51     int array2[ 3 ] = { 1, 2, 3 };
52
53     cout << "\n\nValues on entering automaticArrayInit:\n";
54
55     // output contents of array2
56     for ( int i = 0; i < 3; i++ )
57         cout << "array2[" << i << "] = " << array2[ i ] << " ";
58
59     cout << "\n\nValues on exiting automaticAr";
60
61     // modify and output contents of array2
62     for ( int j = 0; j < 3; j++ )
63         cout << "array2[" << j << "] = "
64             << ( array2[ j ] += 5 ) << " ";
65
66 } // end function automaticArrayInit

```

Automatic array, recreated with every function call.

Although the array is changed, it will be destroyed when the function exits and the changes will be lost.

First call to each function:

Values on entering staticArrayInit:

array1[0] = 0 array1[1] = 0 array1[2] = 0

Values on exiting staticArrayInit:

array1[0] = 5 array1[1] = 5 array1[2] = 5

Values on entering automaticArrayInit:

array2[0] = 1 array2[1] = 2 array2[2] = 3

Values on exiting automaticArrayInit:

array2[0] = 6 array2[1] = 7 array2[2] = 8

Second call to each function:

Values on entering staticArrayInit:

array1[0] = 5 array1[1] = 5 array1[2] = 5

Values on exiting staticArrayInit:

array1[0] = 10 array1[1] = 10 array1[2] = 10

Values on entering automaticArrayInit:

array2[0] = 1 array2[1] = 2 array2[2] = 3

Values on exiting automaticArrayInit:

array2[0] = 6 array2[1] = 7 array2[2] = 8

Passing a Variable By Reference

```
void power(float&, int); // prototype
```

Prototype includes &

```
void main(void)
{
    float base;
    int exp =3;
    cout<< "enter the number whose power you want to find";
    cin >> base;
```

```

    power(base, exp);
    cout <<"result = " <<base ;
}
```

function call is the same

```
void power(float& x, int y)
{
    int product=1;
    for(int j=1; j<=y; j++)
        product=product*x;
    X=product;
}
```

header includes &

Pointers and Referencing

- Reference and pointers have a more or less similar function. However, there are some fundamental differences in how they work.
- Reference is an alias to the variable, while pointer gives the address of the variable.
- References basically function like pointers that are dereferenced every time they are used.

Pointers and Referencing

- However, because you cannot directly access the memory location stored in a reference (as it is dereferenced every time you use the reference), you cannot change the location to which a reference points, whereas you can change the location to which a pointer points.

Passing Arrays by Reference

Function prototype or declaration

```
void somefunct( int [], int);
```

Function call

```
void somefunct(elem, size)
```

Function Header

```
void somefunct( int array[], int  
                size_of_elem)
```

Passing an Array By Reference

```
void display(int [],int);
```

```
int main ( )  
{  
    int num [5] = {23,43,56,34,32};  
    display ( num, 5 );  
}
```

```
void display ( int j[], int n )  
{  
    int i;  
    for (i=0; i<n; i++)  
        cout <<"\n element " <<i <<" = " << j[i];  
}
```

Dynamic Memory Allocation

- Enables allocation and deallocation of memory in a program
- Using 'new' and 'delete' operators
- By using 'new' operator dynamically memory is allocated in the *free store*
- Whereas 'delete' operator deallocated the memory area reserved

Memory Allocation

- There are essentially two types of memory allocation
 - ❑ Static – Done by the compiler automatically (implicitly).
 - ❖ Global variables or objects -- memory is allocated at the start of the program, and freed when program exits; alive throughout program execution
 - Can be access anywhere in the program.
 - ❖ Local variables (inside a routine) – memory is allocated when the routine starts and freed when the routine returns.
 - A local variable cannot be accessed from another routine.
 - ❖ Allocation and free are done implicitly.
 - ❖ No need to explicitly manage memory is nice (easy to work with), but has limitations!
 - ❖ Using static allocation, the array size must be fixed.
 - ❖ Consider the grade roster for the class? What is the number of people in the class?

Memory Allocation

- There are essentially two types of memory allocation
 - ❑ Wouldn't it be nice to be able to have an array whose size can be adjusted depending on needs.
 - ❖ Dynamic memory allocation deals with this situation.
 - ❑ Dynamic – Done explicitly by programmer.
 - ❖ Programmer explicitly requests the system to allocate memory and return starting address of memory allocated (what is this?). This address can be used by the programmer to access the allocated memory.
 - ❖ When done using memory, it must be **explicitly** freed.

Explicitly allocating memory in C++: The 'new' Operator

- Used to dynamically allocate memory
- Can be used to allocate a single variable/object or an array of variables/objects
- The new operator returns pointer to the type allocated
- Examples:
 - `int *my_int_array = new int[20];`
 - Before the assignment, the pointer may or may not point to a legitimate memory
 - After the assignment, the pointer points to a legitimate memory.

Explicitly freeing memory in C++: the 'delete' Operator

- Used to free memory allocated with new operator
- The delete operator should be called on a pointer to dynamically allocated memory when it is no longer needed
- Can delete a single variable/object or an array
 - ❑ delete PointerName;
 - ❑ delete [] ArrayName;
- After delete is called on a memory region, that region should no longer be accessed by the program
- Convention is to set pointer to deleted memory to NULL
 - ❑ Any new must have a corresponding delete --- if not, the program has memory leak.
 - ❑ New and delete may not be in the same routine.

The Heap

- Large area of memory controlled by the runtime system that is used to grant dynamic memory requests.
- It is possible to allocate memory and “lose” the pointer to that region without freeing it. This is called a memory leak.
- A memory leak can cause the heap to become full
- If an attempt is made to allocate memory from the heap and there is not enough, an exception is generated (error)

Examples

□ Example # 1:

```
int * ptr;  
ptr = new int;  
*ptr = 10;  
delete ptr;
```

□ Example # 2:

```
int size;  
cout<<“Enter size of an array”;  
cin>>size;  
int *array;  
array = new int [size];  
delete [] array;
```

Why use dynamic memory allocation?

- Allows data (especially arrays) to take on variable sizes (e.g. ask the user how many numbers to store, then generate an array of integers exactly that size).
- Allows us to create many structures used in Data Structures and Algorithms